Ad-Hoc Overcooked: Collaboration without Pre-Coordination

Daniel Rodrigues
Instituto Superior Técnico
Lisboa, Portugal
daniel.costa.rodrigues@tecnico.ulisboa.pt

Pedro Macedo
pedrosantosmacedo@tecnico.ulisboa.pt
Instituto Superior Técnico
Lisboa, Portugal

Tiago Santos
tiago.castro.santos@tecnico.ulisboa.pt
Instituto Superior Técnico
Lisboa, Portugal

Abstract

Ad hoc teamwork investigates how autonomous agents can cooperate with previously unseen teammates without prior coordination. This project studies this problem in a three-player Overcooked-AI environment, where an ego agent needs to coordinate with two partners whose policies are unknown. We extend the original environment to support three players and compare recurrent self-play with a three-stage ad hoc curriculum. The final policy uses RecurrentPPO with a Multi-Input LSTM architecture and a custom feature extractor. Evaluation is performed against held-out teams designed to test the adapting capability of the ego agent. Results show that the curriculum agent can produce more consistent partner adaptation across test partners, whereas self-play training can achieve stringer but more specialized performances. The passive baseline analysis also shows that the curriculum model contributes actively to the team instead of just relying on its partners to complete the task.

CCS Concepts

• **Computing methodologies** → **Reinforcement learning**; **Multi-agent systems**; • **Human-centered computing** → *Collaborative interaction.*

Keywords

Ad Hoc Teamwork, Zero-Shot Coordination, Multi-Agent Reinforcement Learning, Recurrent PPO, Overcooked-AI

1 Introduction

Ad hoc teamwork studies the problem of designing agents that are capable of cooperating with previously unfamiliar teammates without prior coordination or shared training [3]. This is a critical setting because, in real-life scenarios, we cannot always assume that all agents were trained together under the same conditions. This is especially important when considering robots or software agents that have been programmed by different groups and/or at different times such that it was not known during development that they would need to coordinate. This challenge is defined as collaboration without pre-coordination, where an autonomous agent must infer how to contribute to an existing team and adapt its behavior accordingly [3].

A common issue in cooperative multi-agent reinforcement learning is that teams trained together may develop conventions and strategies that work well in self-play but are brittle when paired with novel partners. This problem has been observed in human-AI coordination environments, where agents trained with similar partners to maximize task performance may not coordinate well with human-like or unknown teammates [1]. For these reasons, the Overcooked-AI environment has become an important benchmark to study coordination; to achieve success, the agents are required to divide tasks, such as collecting ingredients or delivering soups, under spatial and temporal constraints. Recent work has shown that evaluating agents only with familiar partners or fixed layouts is insufficient for measuring zero-shot cooperation ability in the Overcooked Generalization Challenge [2].

In this project, we study ad hoc coordination in a three-player Overcooked environment, where one learning agent, referred to as the ego agent, is paired with two teammates with unknown policies. The main objective is to train an ego policy that can adapt to different team behavior and perform a complementary role. We compare standard self-play training against a three-stage ad hoc curriculum constructed to expose the agent to diverse team compositions and partner behaviors. Our final architecture uses Recurrent PPO with a Multi-Input LSTM policy, allowing the model to make decisions based on information accumulated throughout the episode.

2 Methodology and Implementation

2.1 Problem Formulation

The environment can be modelled as a partially observable Markov decision process. The physical state $s_t \in \mathcal{S}$ contains the kitchen layout, held objects, pot states, and object locations. and the action space is defined as

$$\mathcal{A} = \{\text{North, South, East, West, Interact, Stay}\}$$

Since the partner's policies are unknown, we represent their behaviors as hidden types $c^1, c^2 \in C$. The augmented state space therefore accounts for the hidden teammates behavioral type and is given by $x_t = \langle s_t, c^1, c^2 \rangle \in \mathcal{S} \times C \times C$. At each timestep, the environment executes a joint action:

$$a_t = (a_t^0, a_t^1, a_t^2) \in \mathcal{A}^3 \tag{1}$$

From the ego's perspective, the transition marginalizes over the unknown teammate actions:

$$P(x_{t+1} \mid x_t, a_t^0) = \sum_{\mathbf{a}_t^{-0}} P_{\text{env}}(s_{t+1} \mid s_t, a_t^0, \mathbf{a}_t^{-0}) \prod_{m=1}^{2} \pi_{c^m}(a_t^m \mid s_t). \tag{2}$$

where $\mathbf{a}_t^{-0} = (a_t^1, a_t^2) \in \mathcal{A}^2$ is the teammates' joint action.

The learning agent receives structured observations $o_t^0 = O(s_t, i_t)$, where i_t is the ego index. The policy selects actions using the current observation and its LSTM state, due to the hidden nature of teammate types:

$$a_t^0 \sim \pi_\theta(a_t^0 \mid o_t^0, h_t). \tag{3}$$

The objective is to maximize the expected discounted team return:

$$J(\theta) = \mathbb{E}\left[\sum_{t=0}^{T} \gamma^t r_t\right], \tag{4}$$

where r_t is the shared cooperative reward.

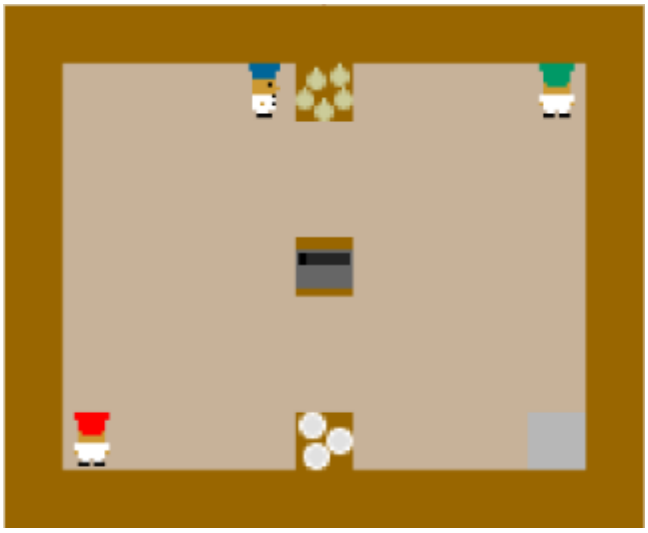

Figure 1: Environment Layout used for training and testing

2.2 Domain and environment

The Overcooked-AI environment is the basis for our project [2]. The environment is a kitchen grid with three chefs whose objective is to prepare and deliver as many onion soups as possible within the episode limit. Each soup requires a fixed sequence of actions: collect three onions, place them into a cooking pot, start cooking, wait until the soup is ready, collect it with a dish, and deliver it at a serving station. The layout contains task-specific tiles such as onion dispensers, cooking pots, dish dispensers, serving stations, counters, and walkable floor tiles. Agents move through the floor tiles and interact with tiles to pick up ingredients, place objects, start cooking, collect soups, or deliver completed orders.

2.3 Three-Agent Extension

The original Overcooked-AI implementation is designed to only support 2 players[1]. Because our objective was to study ad-hoc teamwork capabilities we needed to extend the environment to support one more player. To achieve this, we adapted the environment wrapper so that each episode contained three player slots. At reset, the ego index i.e. the player the ego agent controls is sampled randomly, meaning that the policy can control any of the three available agents. The other two indices are then assigned to partner policies. This design was chosen to prevent the ego policy from overfitting to a specific player position. Instead, the agent must learn a policy that can operate and coordinate from different starting positions and different teammate locations. During each episode, the wrapper constructs a joint action containing the actions of the three players. This is then passed to the Overcooked environment, where all three actions are executed simultaneously. Our implementation, trained models, evaluation scripts, and modified Overcooked-AI code are available in our project repository.[2]

[1]https://github.com/HumanCompatibleAI/overcooked_ai
[2]https://github.com/tiagoc-santos/AASMA

2.4 Architecture

Initially, a PPO agent with a convolutional feature extractor was used. This architecture is well-suited to the spatial structure of Overcooked, as it effectively processes local patterns involving agents, pots, and other objects. Its main advantages lie in its simplicity, faster training and direct representation of the kitchen layout. Nevertheless, this policy does not possess memory: each action is selected based on the current observation. Due to the sequential nature of the Overcooked game, relying only on the current observations limits the adaptation capabilities of the agent since it may need to reason about how partners have behaved over time.

For our final implementation, we used RecurrentPPO with a Multi-Input LSTM policy. The custom feature extractor processes the structured observation through two separate branches. The spatial grid observation, such as agent positions, objects, pot states, counters and dispensers is encoded with a convolutional neural network, because these features depend on local spatial relationships. The joint-action information is processed separately using a small multilayer perceptron, as it is a compact non-spatial vector. The two embeddings are concatenated and projected into a feature representation. This is then passed to the LSTM policy, which maintains an internal state across the episode. The main disadvantage is that the recurrent model is more complex and computationally heavier when compared with its CNN counterpart.

2.5 Training Methodology

All final models were trained for 3,000,000 timesteps, using parallelized environments. We compared two recurrent training procedures: self-play, which was used as the baseline, and a three-stage ad hoc curriculum, used as the proposed training method.

Self-Play baseline. In the self-play procedure, training was divided into ten equal iterations. In the first iteration, the ego agent was paired with two random partners. From the second iteration onward, the current policy was saved to be used as training partners for the next iteration, i.e. at each stage, the ego trained with a previous version of itself.

Scripted partner policies. The ad hoc curriculum used scripted agents to define different partner pools. These included greedy partners, specialist partners, stationary partners, noisy greedy partners and mixed teams. Specialist partners were divided into fetcher agents, which only collect ingredients, and plater agents, which only collect plates and deliver soups. Noisy greedy partners followed the greedy policies with probability $1-\epsilon$ and selected random actions with probability ϵ.

Ad hoc curriculum. The ad hoc curriculum was divided into three stages. The first stage, named pretraining, used 25% of the total timesteps. It serves as an initial learning phase where the agent focuses on developing core mechanics. In this stage, the ego policy was paired with partners that are able to solve the task by themselves, and the dense reward scale was set to 1.0 to encourage the agent to explore the kitchen.

The second stage, role diversity, also consumed 25% of training. In this stage the partner pool is composed of specialist teams, such as plater teams, fetcher teams, stationary specialist combinations and greedy teams. This is the agent's first exposure to teams that

are faulty by nature, and need the ego to perform a specific role. The dense reward scale was reduced to 0.5 to place more emphasis in soup making and delivering and less in intermediate shaping rewards.

The third stage, partner robustness, used the remaining 50% of training. This stage used a broader partner pool containing greedy teams, specialist teams, stationary mixtures, noisy greedy teams, and mixed greedy-noisy teams. Its main purpose is to equip the agent with adapting and inferring skills. The dense reward scale remained at 0.5.

2.6 Evaluation Methodology

After training, the goal was to asses whether the agent was capable of understanding the behavior of 4 held-out partners and adequately adapt to them. Since these teammates were not encountered during training, the agent was forced to infer their underlying policies and adjust its behavior during the evaluation episodes.

The *Alternating Cookers* are partners that perform only two tasks: bringing onions to the cooking pot and starting the cooking process i.e. they do not handle soup delivery. This setting is designed to understand if the agent is capable of realizing that it must take on the task of picking up plates and delivering soup. In contrast, the *Prepositioning Servers* are only capable of picking up plates and delivering soups. This scenario's intent is to evaluate if the agent can take the role of bringing onions to the cooking pot and start the cooking process. With these teams, we evaluate how competent the agent is in taking a fixed role when paired with predictable and unchanging teammates.

The *Role Switchers* initially behave as *Alternating Cookers*. However, after 200 steps, their behavior changes. One of the chefs moves to a corner of the map and becomes stationary, while the other adopts the behavior of a *Prepositioning Server*. This team evaluates how proficient the agent is in understanding and adapting to sudden behavior changes.

Lastly, the *Yielding Generalists* are partners capable of performing every task needed to deliver a soup. When paired with them, the goal is to see how effective the agent is in cooperating with extremely capable partners without disrupting them or relying disproportionately on its partners to complete the task.

To ensure statistical reliability and assess generalization robustness, each model was evaluated over 100 episodes per held-out partner team, with each episode lasting up to 400 environment steps, across two different seeds. The reported results correspond to the average performance across these runs. Given that the chefs' main goal is to deliver as many soups as possible, we use *average_soup_score* as our main metric of evaluation. This value increases by 20 every time a soup is successfully delivered. Other secondary metrics were also collected, such as *avg_time_to_first* and the *avg_coordination_score*. The former shows the average amount of steps spent until the first soup delivery, while the latter is a metric that penalizes the ego agent for standing still and bumping into the environment or its partners:

$$\text{coordination\_score} = \frac{\text{deliveries}}{\text{deliveries} + 0.01 \times \text{bumps} + 0.01 \times \text{stood\_still}}$$

3 Results and Discussion

We evaluate the performance of our proposed Recurrent Proximal Policy Optimization (PPO) agent, trained under the three-stage ad-hoc curriculum, against the standard recurrent self-play baseline.

3.1 Zero-Shot Behavioral Generalization

The foundational objective of this study is to measure how effectively a trained policy adapts to completely unfamiliar teammate combinations without prior coordination. The following graphics summarize the primary performance indicators across the four held-out evaluation partner configurations: *Alternating Cookers*, *Prepositioning Servers*, *Role Switchers*, and *Yielding Generalists*.

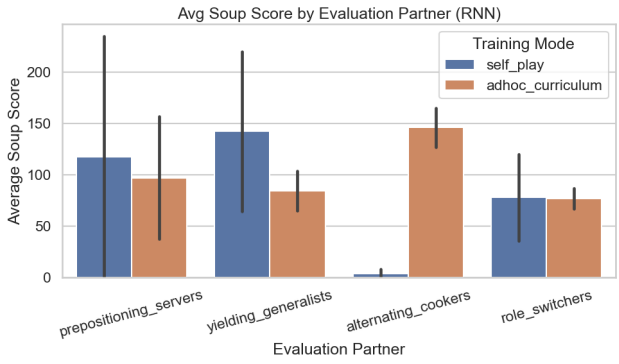

Figure 2: Average Soup Score By Partner Across Evaluation Teams (Bar).

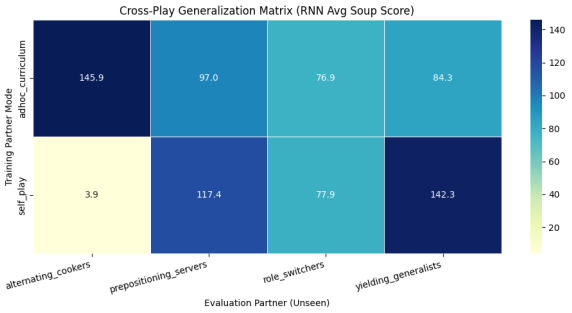

Figure 3: Average Soup Score By Partner Across Evaluation Teams (Cross-Play Matrix).

An initial examination of the cross-play matrix (see Figure 3) can reach an incomplete conclusion. At a glance, the recurrent self-play baseline appears highly effective, achieving peak scores when paired with specific, structurally predictable evaluation teams. However, a closer inspection of the statistical variance exposes severe behavioral volatility. Most critically, it suffers a performance collapse when paired with the Alternating Cookers, a scenario requiring strict role inversion.

In contrast, our proposed Ad-Hoc Curriculum demonstrates more stable generalization capabilities and performance across the held-out teams. It still achieves an overall better average soup

score (101 vs 85) and establishes a higher and more stable performance baseline across all four unseen test teams over the self-play baseline.

3.2 Behavior and coordination analysis

An investigation into the behavioral metrics confirms the curriculum model superior generalization and adaptability.

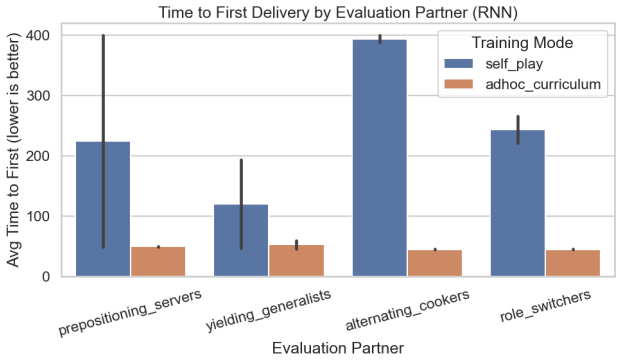

Figure 4: Time to First Delivery Across Evaluation Partners.

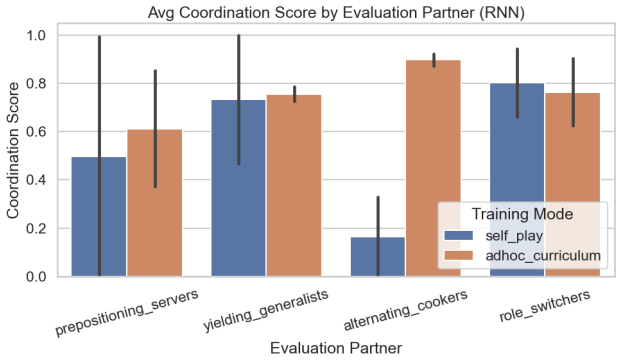

Figure 5: Average Coordination Score Across Evaluation Partners.

The *Time to First Delivery* (see Figure 4) is an indicator for how quickly an agent figures out its teammates' underlying policies. The curriculum agent synchronization is remarkably rapid, establishing a stable delivery pipeline within approximately 50 to 55 timesteps across all unseen teams. It also outscores the self-play agent in coordination score, indicating more fluid behavior with less collisions and unnecessary idling (see Figure 5).

3.3 Marginal Contribution and Passive Baseline

To demonstrate that the ego agent actively drives the team's cooperative output rather than merely being carried by efficient heuristic partners and based on the evaluation technique proposed by Stone et al. [3], we calculate its *Marginal Contribution*. This metric isolates the agent's net added value by taking the difference between the full team score and a passive baseline where the ego agent is replaced by a stationary chef.

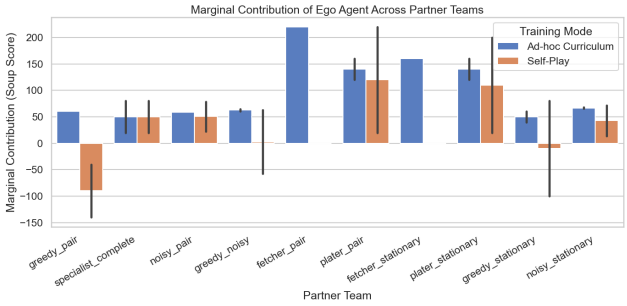

Figure 6: Marginal Contribution Across Evaluation Teams.

The results expose a major flaw in self-play. When paired with standard, moving teams, the self-play agent frequently shows a negative marginal contribution. This means the team actually performs better when the self-play agent stands completely still, suggesting that its learned conventions can interfere with some teammate behaviors (see Figure 6).

Our Ad-hoc Curriculum agent, conversely, consistently adds positive value to every single team (see Figure 6). The impact is clearest when it is paired with incomplete or highly specialized teams that are missing a critical role (such as partners that only fetch onions or only plate food). In these situations, the passive baseline score is exactly zero because the partners cannot physically finish a soup alone. Our curriculum agent appears to select a complementary role in this scenarios, stepping in to fill the missing role, and significantly raises the final score. This suggests that the curriculum training encourages more flexible ad hoc behavior than self-play in the tested scenarios.

4 Limitations and Critical Analysis

Although the proposed ad hoc curriculum improves generalization across the held-out partner teams, it is not without flaws.

4.1 Limitations of the Work

Both the agent's training and evaluation are performed on a single layout. Since the locations of the tiles are fixed, the agent may learn movement patterns specific to this kitchen, rather than strategies that can be transferred to other layouts. Additionally, the team's size is set to 3 chefs across the entire project. As such, the agent's learned behaviors might be strongly tied to the assumption that it will be paired with two other chefs, making it unreliable in larger or even smaller teams.

4.2 Limitations of the Solutions Provided

The proposed solution is heavily reliant on a small pool of partners with manually designed behaviors, rather than separately trained agents or human-derived policies. As such, the diversity of teammates encountered both in training and evaluation is limited by the behaviors explicitly programmed, which might lead to a case of overfitting. The lack of unpredictable human behavior in the partners is extremely relevant in an ad hoc teamwork setting, since real human teammates would likely display inconsistent and unexpected behavior, rather than strictly following a scripted policy.

4.3 Limitations of the Results

The passive baseline used to calculate a marginal contribution is an extreme comparison. Replacing the ego agent with a stationary chef is useful because it shows whether the trained agent contributes more than relying disproportionately on its partners. Nevertheless, it is a weak baseline when compared with a real teammate or even a partially competent policy.

5 Conclusion and Future Work

This project studied ad hoc teamwork in a three-player Overcooked setting, comparing recurrent self-play with a three-stage ad hoc curriculum. The results show that training with recurrent memory can produce more consistent partner adaptation across held-out teammate behaviors, whereas self-play can achieve strong but more specialized performances. The passive baseline analysis also shows that the curriculum-trained agent contributes actively to the team instead of just being carried by its partners. Future work should extend this approach by evaluating the curriculum agent in more complex layouts, introducing behavior-cloned partners or human players to obtain a more realistic test setting and by enhancing the current architecture by adding a specific partner-modeling module, such as a role classifier or an attention mechanism over teammates, to allow the agent direct reasoning about partner behavior.

AI Usage and Methodology

During this assignment, we used LLMs as auxiliary tools. They were used to support code writing, debugging and understanding of implementation details, particularly for the 3 players extension. However, all generated suggestions were manually reviewed, tested and adapted to the project requirements before being included in the final code.

References

[1] Micah Carroll, Rohin Shah, Mark Ho, Tom Griffiths, Sanjit Seshia, Pieter Abbeel, and Anca Dragan. 2019. On the Utility of Learning about Humans for Human-AI Coordination. In *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (Eds.), Vol. 32. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2019/file/f5b1b89d98b7286673128a5fb112cb9a-Paper.pdf

[2] Constantin Ruhdorfer, Matteo Bortoletto, Anna Penzkofer, and Andreas Bulling. 2024. The overcooked generalisation challenge. (2024).

[3] Peter Stone, Gal A. Kaminka, Sarit Kraus, and Jeffrey S. Rosenschein. 2010. Ad Hoc Autonomous Agent Teams: Collaboration without Pre-Coordination. In *Proceedings of the Twenty-Fourth Conference on Artificial Intelligence.*